BAGIRA

Technical Test – Full-Stack developer (Frontend oriented)

Dear candidate,

As part of the recruitment process at Bagira Systems, you are required to complete this assignment. You will receive the assignment at the time of your choice (e.g., 9 AM) and must submit your solution within 24 hours (e.g., by 9 AM the next day) ***please read the description carefully before starting.***

Objectives:

Implement missing parts in a provided code skeleton (backend and frontend) for a small web application used to create training scenarios and manage entities inside each scenario

Instruction:

1. Clone this repository – Bagira-Assignment and create a fork with the following structure – 'BagiraAssignment_<YourName>'
2. Make sure to work on the folder named **'Frontend-Oriented'**.
3. The backend contains pre-implemented domain models, interfaces, and in-memory repositories. You must implement the API controllers and complete the backend wiring.
4. Choose either the React or Angular frontend skeleton in the `frontend/` folder and build your application from there. Specify which one you chose.
5. Follow the **README** instructions on the backend and frontend folders for detailed instructions.
6. Implement the missing functionality marked in the code (TODOs) and ensure the app works end-to-end.

Requirements:

1. Data rules:
    - EntityType must be one of the following - Soldier, Tank, Drone, Aircraft, Vehicle, Civilian.
    - TaskForce must be one of the following - Friendly, Enemy.
    - Latitude must be between -90 and 90.
    - Longitude must be between -180 and 180.
    - Entities always belong to exactly one Scenario (no orphan entities).
2. Screens:
    - Scenario List – view all scenarios, navigate to details, create.
    - Create Scenario – the form should contain name and description.
    - Scenario Details – view scenario info and manage its entities.
    - Create Entity – the form must include:
        - EntityType
        - TaskForce
        - Name/Callsign
        - Latitude
        - Longitude
3. Backend:
    - Already provided:
        - Domain models: `Scenario`, `Entity`, `EntityType`, `TaskForce`
        - Repository interfaces: `IRepository`, `IScenarioRepository`, `IEntityRepository`
        - Repository implementations: `ScenarioRepository`, `EntityRepository` (in-memory)
        - DTO structure: `CreateScenarioRequest`, `CreateEntityRequest`, `ScenarioListItemDto`, `ScenarioDetailsDto`, `EntityDto`, `ErrorResponse`
    - You must implement:
        - API Controllers – implement all endpoint stubs in `ScenariosController` and `EntitiesController`.
        - Error Handling – proper HTTP status codes; return `ErrorResponse` on 400 and 404.
        - CORS – configure CORS in `Program.cs` to allow requests from your frontend origin.
4. Frontend - Build your application inside the chosen skeleton (`frontend/frontend-react` or `frontend/frontend-angular`).
    - Implement all four required screens.
    - Build a service/API abstraction layer (architecture is your decision).
    - Form submission with real-time validation and display of server-side error messages.
    - Filtering: scenarios by name/description, entities by EntityType and TaskForce.

- Loading states, empty states, and error states on all data-fetching views.

5. Docker:
   - After implementing the core functionality, containerize the solution using Docker:
     i. Create Docker files for Backend and Frontend
     ii. Create a docker-compose.yml.
     iii. Ensure the solution runs with: docker-compose up
     iv. Update the main README with Docker instructions
   - Notes:
     i. Use multi-stage builds for frontend (build + serve)
     ii. Configure CORS in backend to allow frontend origin
     iii. Use environment variables for API base URL in frontend
     iv. Document any required environment variables
     v. **ONLY If Docker takes more time than expected**, write a short explanation in the README describing how you would containerize the solution and move on.
6. Optional bonus:
   - Connect to a database of your choice (replace in-memory repositories).
   - Map view for entities — display entities on a map using their coordinates.
   - Global search — search across scenarios and entities from a single search bar.
   - Server-side sorting for scenarios and entities.
   - Add tests for validation (backend) or key UI flows (frontend).
   - Implement Update and Delete for Scenarios and Entities end-to-end.
   - UX improvements.

Acceptance criteria:

- User can create a scenario and see it in the scenario list.
- User can open scenario details and view entities belonging to that scenario only.
- User can select TaskForce (Friendly/Enemy) when creating entities, and TaskForce is displayed in entity lists.
- User can filter and sort entities by Type, TaskForce, and other fields.
- User can search for scenarios and entities.
- User can view entities on a map and toggle between table and map views.
- Toast notifications appear for successful operations and errors.
- Frontend validates all input fields (coordinates, types, required fields).
- Solution runs locally.
- Using clean architecture.
- Code quality and maintainability (structure, naming, separation of concerns).
- Validation and error handling.
- Reasonable scope management within the timebox.
- *** Solution runs in Docker using docker-compose up with all services properly configured and communicating.

Please provide:

- A link to your branch.
- Short notes (bullet points) describing what you completed and any trade-offs.

Good Luck!